



Министерство науки и высшего образования Российской Федерации
Калужский филиал
федерального государственного автономного
образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУК «Информатика и управление»

КАФЕДРА ИУК4 «Программное обеспечение ЭВМ, информационные технологии»

ЛАБОРАТОРНАЯ РАБОТА №3

«JAVASCRIPT»

ДИСЦИПЛИНА: «Кроссплатформенная разработка программного обеспечения»

Выполнил: студент гр. ИУК4-43Б _____ (____Афанасьева Ю.Р.____)
(Подпись) (Ф.И.О.)

Проверил: _____ (____Бибиков А.П.____)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:

- Оценка:

Цель работы: получить навык разработки с использованием языка программирования JavaScript.

Задачи:

1. Разработать JS-скрипт, реализующий действие на странице в браузере согласно варианту
2. Добавить скрипт к странице.

Вариант 3

Задание: разработать браузерную игру «Змейка».

Этапы выполнения работы:

1. Создана HTML-страница, подключены отдельные файлы стилей CSS и сценария JavaScript с атрибутом defer.
2. На страницу добавлен элемент Canvas размером 600×600 пикселей. В JavaScript получен двумерный контекст Canvas и задан размер одной игровой клетки.
3. Змейка представлена массивом объектов с координатами x и y. Реализована её отрисовка на Canvas с помощью метода fillRect() и цикла for.
4. Реализованы очистка игрового поля методом clearRect(), перемещение змейки путём добавления новой головы и удаления хвоста, а также игровой цикл с использованием setTimeout().
5. Добавлено управление клавишами со стрелками с помощью обработчика события keydown. Реализован запрет разворота в противоположную сторону и изменения направления более одного раза за игровой шаг.
6. Реализованы проверки столкновения змейки с границами игрового поля и собственным телом. При столкновении игровой цикл завершается и выводится сообщение о причине проигрыша.
7. Добавлена еда, её отрисовка и генерация в случайной свободной клетке. При совпадении координат головы и еды змейка увеличивается на одну клетку.
8. Добавлен счёт, увеличивающийся после поедания еды, а также рекорд, сохраняемый в локальном хранилище браузера localStorage.
9. Разработан интерфейс запуска и повторного запуска игры. Кнопка размещена в центре игрового поля, а после проигрыша рядом с ней отображается сообщение.
10. Выполнена проверка запуска игры, управления змейкой, роста, генерации еды, подсчёта очков, столкновений и повторного запуска.

Листинг программы:

Файл index.html:

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Змейка</title>
  <link rel="stylesheet" href="style.css">
  <script src="script.js" defer></script>
</head>

<body>
  <h2>Игра «Змейка»</h2>
  <p id="cnt-text">
    Счёт: <span id="cnt">0</span> |
    Рекорд: <span id="record">0</span>
  </p>
  <div class="game-field">
    <canvas id="game-canvas" width="600" height="600">
      Ваш браузер не поддерживает Canvas.
    </canvas>
    <div class="game-controls">
      <p id="game-message" hidden></p>
      <button id="restart-button" type="button">Начать игру</button>
    </div>
  </div>
</body>
</html>
```

Файл script.js:

```
const canvas = document.getElementById("game-canvas");
const context = canvas.getContext("2d");
const cntElement = document.getElementById("cnt");
const recordElement = document.getElementById("record");
const restartButton = document.getElementById("restart-button");
const gameMessage = document.getElementById("game-message");
const cellSize = 20; // размер одной клетки
let cnt = 0;
let record = Number(localStorage.getItem("snakeRecord")) || 0;
recordElement.textContent = record;
const snake = [
  { x: 10, y: 10 },
  { x: 9, y: 10 },
  { x: 8, y: 10 }
];

// начальные координаты еды
const food = { x: 20, y: 15 };

// Вправо: directionX = 1, directionY = 0
```

```

// Влево: directionX = -1, directionY = 0
// Вниз: directionX = 0, directionY = 1
// Вверх: directionX = 0, directionY = -1

// начальное направление
let directionX = 1;
let directionY = 0;
let directionChanged = false;

function drawSnake() {
  context.fillStyle = "green";
  for (let i = 0; i < snake.length; i++) {
    const part = snake[i];
    context.fillRect(part.x * cellSize, part.y * cellSize, cellSize, cellSize);
  }
}

function drawFood() {
  context.fillStyle = "red";
  context.fillRect(food.x * cellSize, food.y * cellSize, cellSize, cellSize);
}

function clearCanvas() {
  context.clearRect(0, 0, canvas.width, canvas.height);
}

function moveSnake() {
  const head = snake[0]; // первый элемент массива - голова змейки
  const newHead = {
    x: head.x + directionX,
    y: head.y + directionY
  };
  snake.unshift(newHead); // добавляет новую голову в начало массива

  if (newHead.x === food.x && newHead.y === food.y) { // если координаты головы
    совпали с едой, то хвост не удаляем и генерируем еду в рандомном месте
    cnt = cnt + 1;
    cntElement.textContent = cnt;

    if (cnt > record) {
      record = cnt;
      recordElement.textContent = record;
      localStorage.setItem("snakeRecord", record);
    }
  }

  const columns = canvas.width / cellSize;
  const rows = canvas.height / cellSize;
  let foodOnSnake = true;

  // повторяет генерацию, если случайная клетка занята змейкой
  while (foodOnSnake) {

```

```

    food.x = Math.floor(Math.random() * columns);
    food.y = Math.floor(Math.random() * rows);
    foodOnSnake = false;

    for (let i = 0; i < snake.length; i++) {
        if (snake[i].x === food.x && snake[i].y === food.y) {
            foodOnSnake = true;
            break;
        }
    }
} else {
    snake.pop(); // удаляет хвост
}
}

function changeDirection(event) {
    // До следующего перемещения второе изменение направления запрещено.
    if (directionChanged) {
        return;
    }

    if (event.key === "ArrowUp" && directionY === 0) {
        directionX = 0;
        directionY = -1;
        directionChanged = true;
    } else if (event.key === "ArrowDown" && directionY === 0) {
        directionX = 0;
        directionY = 1;
        directionChanged = true;
    } else if (event.key === "ArrowLeft" && directionX === 0) {
        directionX = -1;
        directionY = 0;
        directionChanged = true;
    } else if (event.key === "ArrowRight" && directionX === 0) {
        directionX = 1;
        directionY = 0;
        directionChanged = true;
    }
}

// вызывает changeDirection при каждом нажатии клавиши
document.addEventListener("keydown", changeDirection);

function gameLoop() {
    clearCanvas();
    moveSnake();
    directionChanged = false;

    const head = snake[0];
    if (head.x < 0 || head.y < 0 || head.x * cellSize >= canvas.width || head.y *
cellSize >= canvas.height) {

```

```

        restartButton.hidden = false;
        gameMessage.textContent = "Игра окончена: змейка столкнулась со стеной";
        gameMessage.hidden = false;
        return;
    }

    // проверяет, не совпали ли координаты головы с одной из частей тела
    for (let i = 1; i < snake.length; i++) {
        if (head.x === snake[i].x && head.y === snake[i].y) {
            restartButton.hidden = false;
            gameMessage.textContent = "Игра окончена: змейка столкнулась с собой";
            gameMessage.hidden = false;
            return;
        }
    }

    drawSnake();
    drawFood();
    setTimeout(gameLoop, 300);
}

drawSnake();
drawFood();

// при нажатии кнопки возвращает начальные данные и запускает игру
restartButton.addEventListener("click", function () {
    snake.length = 0;
    snake.push(
        { x: 10, y: 10 },
        { x: 9, y: 10 },
        { x: 8, y: 10 }
    );

    directionX = 1;
    directionY = 0;
    directionChanged = false;
    food.x = 20;
    food.y = 15;
    cnt = 0;
    cntElement.textContent = cnt;

    clearCanvas();
    drawSnake();
    drawFood();
    gameMessage.hidden = true;
    restartButton.hidden = true;
    setTimeout(gameLoop, 300);
});

```

Файл style.css:

```
body {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  background-color: #0b1b34;
  color: white;
}

h2 {
  margin-bottom: 10px;
}

#cnt-text {
  margin-top: 0;
  font-size: 20px;
}

.game-field {
  position: relative;
}

canvas {
  display: block;
  border: 2px solid black;
  background-color: #eeee;
}

.game-controls {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 15px;
  width: 80%;
  text-align: center;
}

#restart-button {
  padding: 18px 35px;
  border: 2px solid white;
  border-radius: 10px;
  background-color: #0b1b34;
  color: white;
  font-size: 22px;
  cursor: pointer;
}
```

```

}

#game-message {
  margin: 0;
  padding: 12px 18px;
  border-radius: 8px;
  background-color: #0b1b34;
  color: white;
  font-size: 20px;
  text-align: center;
}

```

Результаты работы программы представлены на рисунках 1-7:

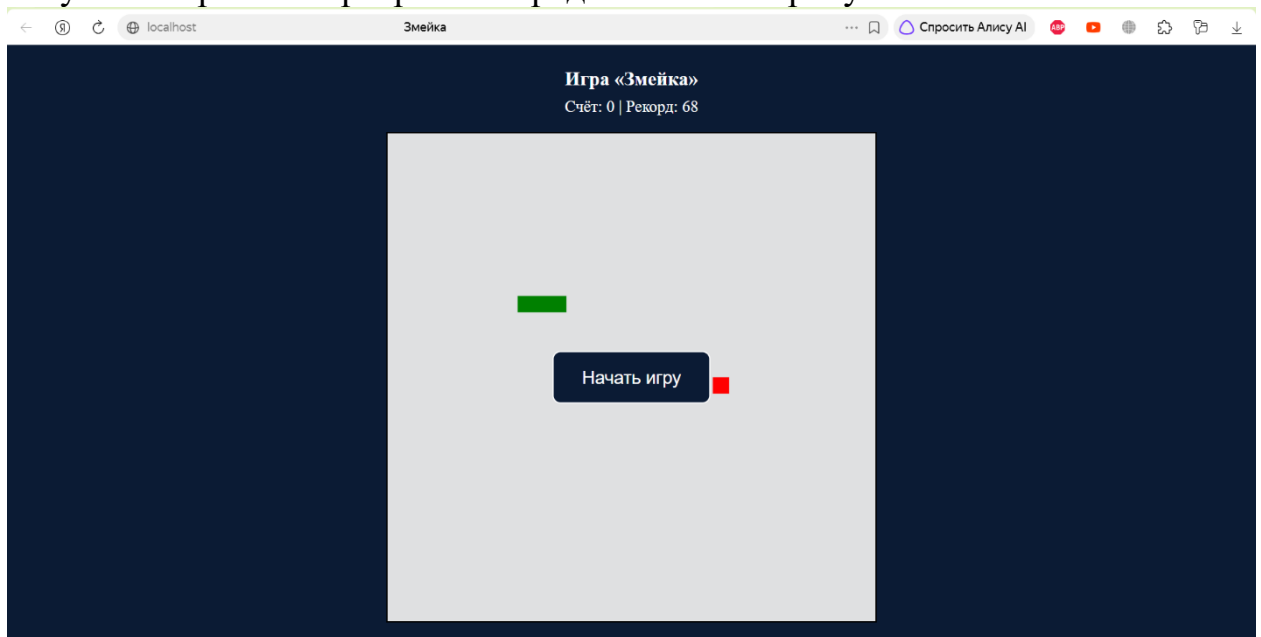


Рисунок 1



Рисунок 2

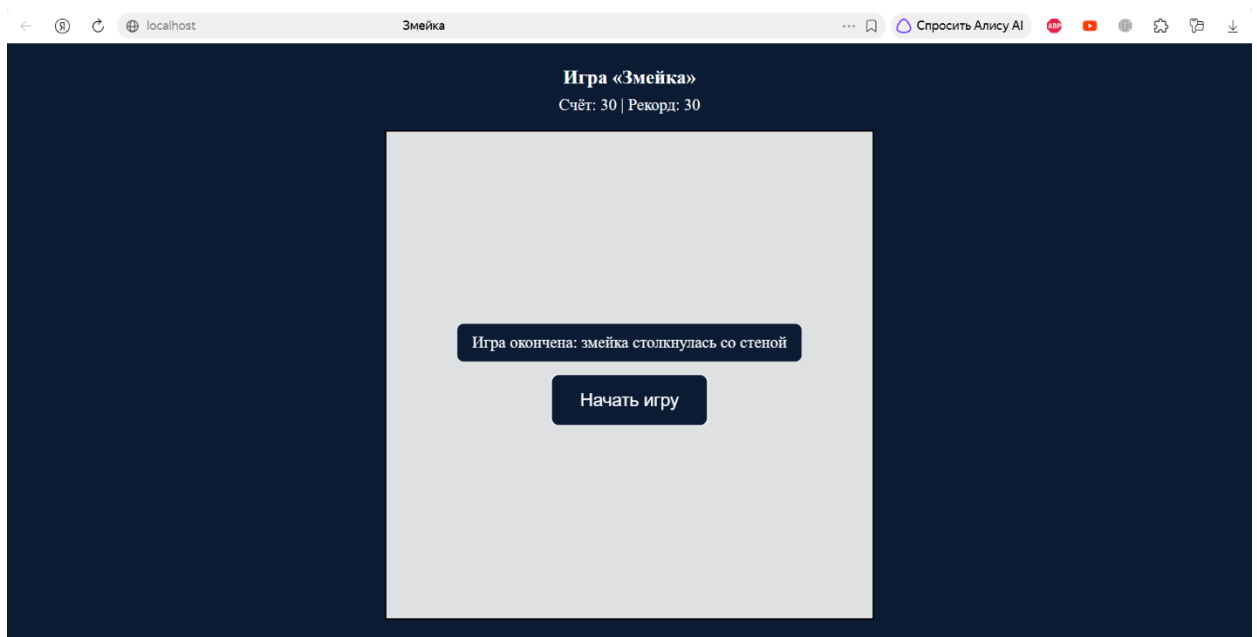


Рисунок 3

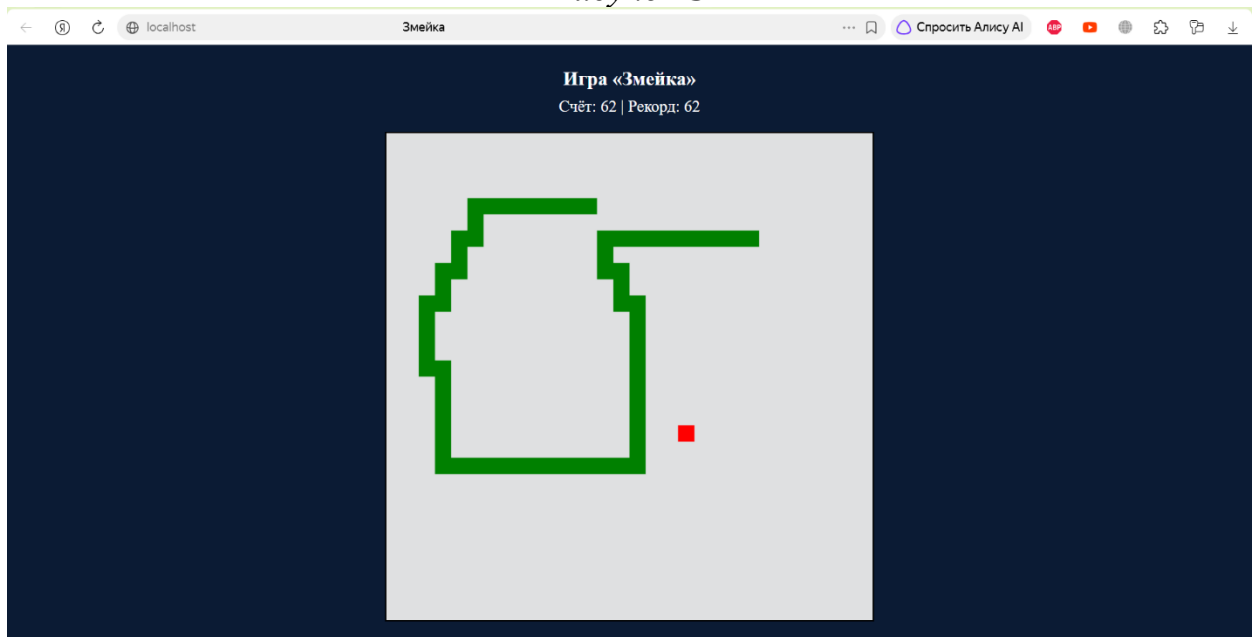


Рисунок 4



Рисунок 5

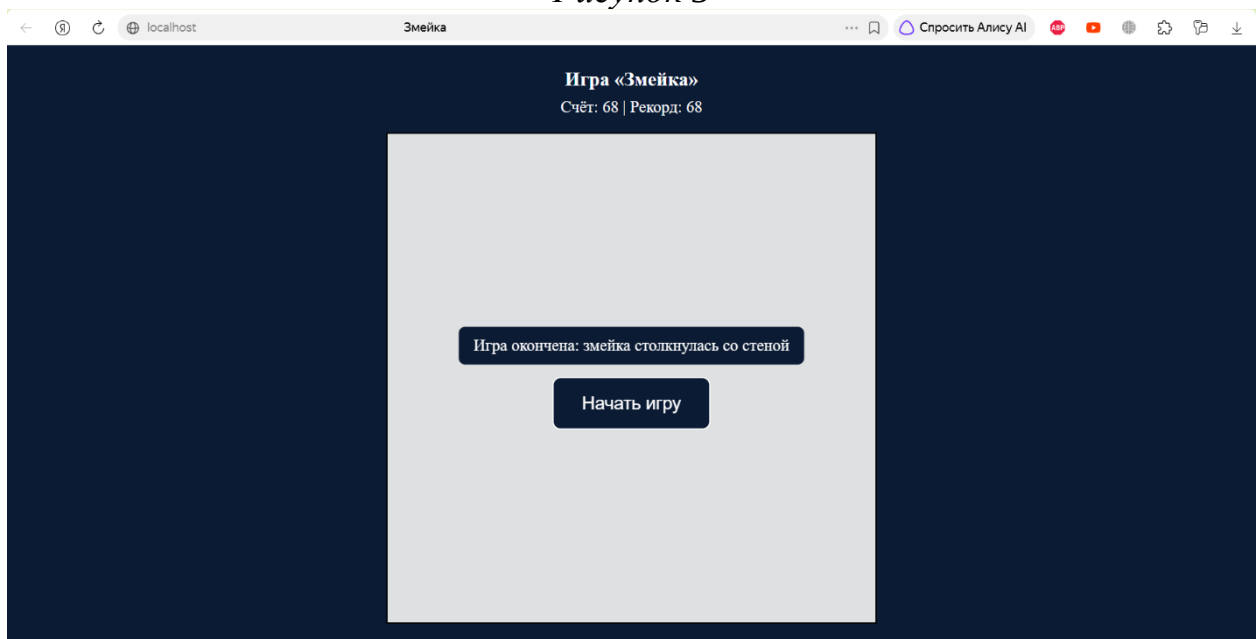


Рисунок 6

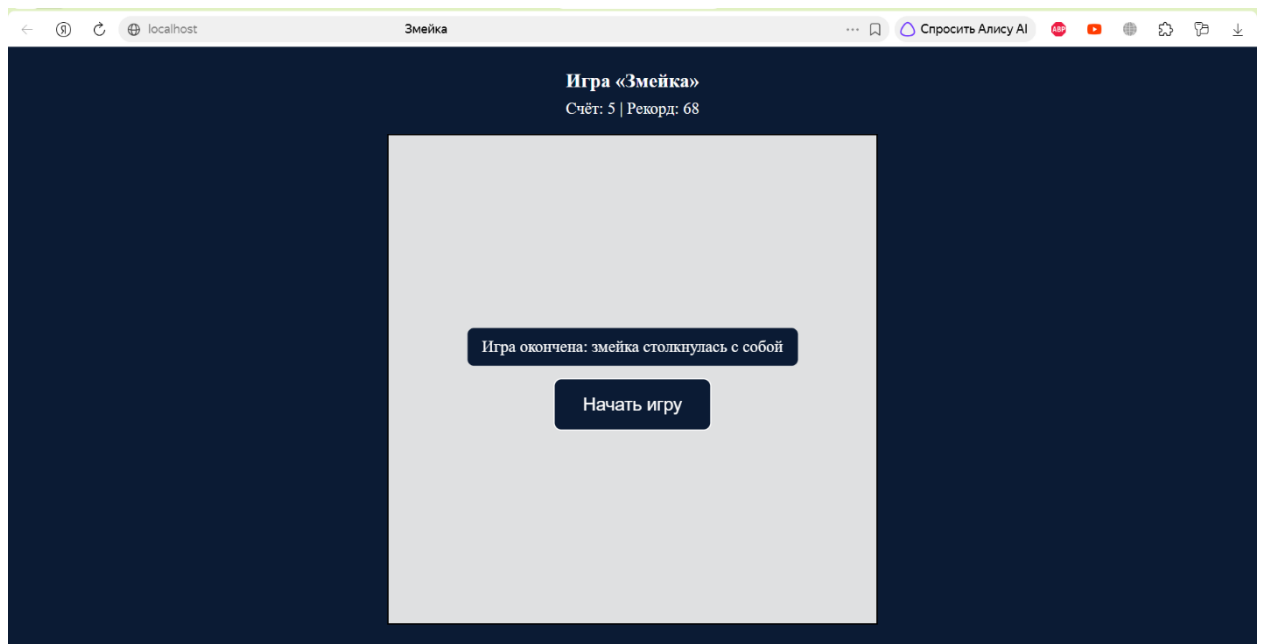


Рисунок 7

Выводы: в ходе лабораторной работы были получены практические навыки разработки с использованием языка программирования JavaScript.